

1. Core Building Blocks

- **Activity:** Represents a single screen with user interface
Example: Login screen, Home screen
- **Service:** Runs in background without user interaction
Example: Music playing in background
- **Broadcast Receiver:** Responds to system-wide messages
Example: Battery low notification
- **Content Provider:** Manages and shares app data between applications
Example: Sharing contacts between apps
- **Example:** In a WhatsApp-like app: Activity = chat screen, Service = message sync, Broadcast Receiver = network change

2. AndroidManifest.xml

- **Definition:** Main configuration file of Android application
- **Purpose:** Provides essential information about the app to the system
- **Key Components**
 - Declares all activities
 - Defines permissions (Internet, Camera)
 - Specifies app name, icon
 - Declares services and receivers
- **Structure (Basic)**

```
<manifest>
  <application>
    <activity android:name=".MainActivity">
      <intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category
android:name="android.intent.category.LAUNCHER"/>
      </intent-filter>
    </activity>
  </application>
</manifest>
```

Example: If app

3. Basic UI Widgets

Definition: UI elements used to build user interface

- **Common Widgets**
 - **TextView** – displays text
 - **EditText** – input field
 - **Button** – user action
 - **ImageView** – display images
 - **CheckBox / RadioButton** – selection options
- **Features**
 - Defined in XML layout
 - Can be controlled using Java/Kotlin

Example: Login form uses

4. Activity

- **Definition:** Activity is a single screen in Android app

- **Lifecycle Methods**
 - **onCreate()** – initialization
 - **onStart()** – visible to user
 - **onResume()** – user interaction starts
 - **onPause()** – partially hidden
 - **onStop()** – not visible
 - **onDestroy()** – activity destroyed
 - **Working:** Activity starts when user opens app or navigates
 - **Example:** Opening settings screen creates a new activity
-

5. Layout

- **Definition:** Layout defines structure of UI elements
 - **Types of Layouts**
 - **LinearLayout** – arranged in row/column
 - **RelativeLayout** – position relative to other views
 - **ConstraintLayout** – flexible positioning
 - **FrameLayout** – single view container
 - **Features**
 - **Defined using XML**
 - **Controls placement and alignment**
 - **Example:** Login screen arranged using **LinearLayout** vertically
-

6. Intent

- **Definition:** Intent is used to communicate between components
 - **Types**
 - **Explicit Intent** – specify exact component
 - **Implicit Intent** – system decides component
 - **Uses**
 - **Start activity**
 - **Start service**
 - **Send data between components**
 - **Example:** Opening camera using implicit inten
-

7. Fragments

- **Definition:** Fragment is a reusable part of UI inside an activity
- **Features**
 - **Has its own lifecycle**
 - **Can be combined in one activity**
 - **Used for dynamic UI**
- **Advantages**
 - **Reusability**
 - **Better UI design for tablets**
- **Example:** In email app: list on left, content on right using fragments.

Basic UI Components – Definitions¹

1. TextView: TextView is a UI component used to display text to the user

- **Features**
 - Displays static or dynamic text
 - Supports formatting (color, size, style)
 - **Example:** Showing “Welcome User” on screen
-

2. EditText : EditText is a UI component used to accept input from the user

- **Features**
 - Allows typing text
 - Supports input types (text, number, password)
 - **Example:** Entering username and password in login form⁵
-

3. Spinner: Spinner is a dropdown list that allows user to select one option

- **Features**
 - Displays selected item
 - Shows list on click
 - **Example**
Selecting country from dropdown list
-

4. DatePicker Dialog: DatePicker Dialog is used to select a date using a calendar interface

- **Features**
 - User-friendly calendar view
 - Prevents invalid date input
 - **Example:** Selecting date of birth
-

5. TimePicker Dialog: TimePicker Dialog is used to select time (hours and minutes)

- **Features**
 - Supports 12/24-hour format
 - Easy time selection
 - **Example:** Setting alarm time
-

6. Button: Button is a clickable UI component used to perform an action

- **Features**
 - Triggers events on click
 - Can display text or icon
 - Example:** Submit button in form
-

7. ToggleButton: ToggleButton is a button with two states (ON/OFF)

- **Features**
 - Represents binary choice
 - Changes state on click
- **Example:** Turning WiFi ON/OFF

Basic UI Components – Definitions2

8. ImageButton: **ImageButton** is a button that displays an image instead of text

- **Features**
 - Used for icons
 - Clickable like normal button
- **Example:** Camera icon button

9. RadioButton: **RadioButton** allows user to select one option from multiple choices

- **Features**
 - Used inside **RadioGroup**
 - Only one selection allowed
- **Example:** Selecting gender (Male/Female)

10. RadioGroup: **RadioGroup** is a container that groups multiple **RadioButtons**

- **Features**
 - Ensures only one option is selected
 - Manages **RadioButton** behavior
- **Example:** Group of options for payment method

11. CheckBox: **CheckBox** allows user to select multiple options

- **Features**
 - Multiple selections allowed
 - Independent choices
- **Example:** Selecting hobbies (music, sports, reading)

12. AutoCompleteTextViewAu: **toCompleteTextView** provides suggestions while user types

- **Features**
 - Shows dropdown suggestions
 - Improves typing speed
- **Example:** Search bar showing suggestions

13. MultiAutoCompleteTextView: **MultiAutoCompleteTextView** allows multiple auto-complete inputs

- **Features**
 - Supports multiple entries
 - Uses separators (comma, space)
- **Example:** Entering multiple email addresses

14. TextInputLayout: **TextInputLayout** is a Material Design wrapper for **EditText**

- **Features**
 - Floating label
 - Error message display
- **Example:** Input field with animated label

Advanced Views1

1. CardView: CardView is a UI component used to display content inside a card-like layout

- **Features**
 - Provides elevation and shadow
 - Used for modern UI design
 - **Example:** Product displayed in card format in shopping app
-

2. RecyclerView: RecyclerView is an advanced and efficient view used to display large datasets

- **Features**
 - Reuses views (ViewHolder pattern)
 - Supports multiple layouts
 - **Example:** News feed or product list
-

3. ListView: ListView is a view used to display items in a vertical scrollable list

- **Features**
 - Simple implementation
 - Uses adapter for data
 - **Example:** Contact list
-

4. GridView: GridView displays items in a two-dimensional grid (rows and columns)

- **Features**
 - Suitable for images
 - Uses adapter
 - **Example:** Image gallery
-

5. ScrollView: ScrollView is a layout that allows vertical scrolling of content

- **Features**
 - Used when content exceeds screen size
 - Contains single child layout
 - **Example:** Long form or article page
-

6. WebView : WebView is used to display web pages inside an Android app

- **Features**
 - Supports HTML, CSS, JavaScript
 - Acts like mini browser
- **Example:** Opening website inside app

Advanced Views2

7. SearchView:

SearchView is a UI component used to perform search operations

- **Features**
 - Provides search input field
 - Supports suggestions
 - **Example:** Searching products in app
-

8. TabLayout: **TabLayout** is used to organize content into tabs

- **Features**
 - Easy navigation between screens
 - Works with ViewPager
 - **Example:** Tabs like Home, Profile, Settings
-

9. ExpandableListView:

ExpandableListView shows items in expandable and collapsible groups

- **Features**
 - Parent-child structure
 - Saves screen space
 - **Example:** FAQ section
-

10. DynamicListView:

DynamicListView is a list whose data can be added, removed, or updated at runtime

- **Features**
 - Real-time updates
 - Works with adapter
- **Example:** Chat messages updating dynamically.

❖ **RecyclerView with Adapter :**

RecyclerView uses adapter for efficient data display

- **Components**
 - ViewHolder – holds item views
 - Adapter – binds data
 - LayoutManager – defines layout
- **Working**
 - Reuses views to improve performance
 - Suitable for large datasets
- **Example:** News app displaying articles list using RecyclerView.

Adapters in Android

1. ArrayAdapter: **ArrayAdapter** is a bridge between an array or list of data and a UI component like **ListView** or **Spinner**

- **Working**
 - Takes data from array/list
 - Converts each item into a view
 - Displays items in UI
- **Features**
 - Simple and easy to use
 - Uses predefined layouts like `simple_list_item_1`
 - Used for static or small datasets
 - Can be customized with custom XML layout
- **Example:** **Displaying list of student names in ListView using String array**

2. SimpleCursorAdapter: **SimpleCursorAdapter** is used to bind data from a database (**Cursor**) to UI components

- **Working**
 - Fetches data using **Cursor (SQLite database)**
 - Maps database columns to UI views
- **Features**
 - Suitable for database-driven apps
 - Works directly with **SQLite queries**
 - Automatically handles data mapping
 - Requires column-to-view ID mapping
- **Example:** **Displaying contacts stored in SQLite database in ListView**

3. BaseAdapter: **BaseAdapter** is a custom adapter used when default adapters are not sufficient

- **Working**
 - Developer overrides methods like `getView()`
 - Custom layout is created for each item
- **Important Methods**
 - `getItem()` – returns item
 - `getView()` – creates and returns view
 - `getCount()` – returns total items
- **Gives full control over item UI and data binding**
- **Commonly used with ViewHolder pattern for performance**
- **Features**
 - Highly flexible
 - Used for complex UI designs
- **Example:** **Creating custom list with image and text in each row.**

Layouts in Android1

1. ConstraintLayout: ConstraintLayout is a flexible layout that positions UI elements using constraints relative to other views or parent

- **Working**
 - Each view is connected using constraints (top, bottom, start, end)
 - Allows complex UI with flat hierarchy
 - **Features**
 - Improves performance
 - Reduces nested layouts
 - **Example:** Login screen with aligned fields using constraints
-

2. LinearLayout: LinearLayout arranges UI elements in a single direction (horizontal or vertical)

- **Working**
 - Uses orientation property
 - Places views one after another
 - **Features**
 - Simple and easy to use
 - Supports weight for space distribution
 - **Example:** Vertical arrangement of username, password, button
-

3. TableLayout: TableLayout organizes UI elements in rows and columns like a table

- **Working**
 - Uses TableRow for each row
 - Columns are automatically aligned
 - **Features**
 - Suitable for structured data
 - Flexible column arrangement
 - **Example:** Displaying marksheet in tabular form
-

4. FrameLayout : FrameLayout is a simple layout that displays one view at a time

- **Working**
 - Views are placed on top of each other
 - Last added view appears on top
- **Features**
 - Used for fragments
 - Suitable for overlays
- **Example:** Displaying a fragment inside activity

Layouts in Android2

5. RelativeLayout: **RelativeLayout positions UI elements relative to each other or parent**

- **Working**
 - **Uses rules like below, above, toRightOf**
 - **Views depend on positions of other views**
- **Features**
 - **Reduces nesting compared to LinearLayout**
 - **Flexible positioning**
- **Example:** **Button placed below TextView**

6. Custom Layout: **Custom Layout is a user-defined layout created by combining or extending existing layouts**

- **Working**
 - **Developer designs layout as per requirement**
 - **Can extend View or ViewGroup**
- **Features**
 - **Provides full control over UI**
 - **Used for unique designs**
- **Example:** **Custom chat bubble layout in messaging app.**